

1 Properties of the RSA algorithm:

1.1 Explain the demonstration of the RSA algorithm

(Encryption/Asymmetric/Demonstration RSA):

GOALS:

- We want a public key (E, N) and a private key D so anyone can encrypt a message M and only the holder of D can recover M .
- Work in modular arithmetic modulo N . **A message block must satisfy $0 \leq M < N$.** If messages are longer, split into blocks or use hybrid encryption.

RELIES IN THE PROPERTIES OF:

- Prime numbers (numbers that only have integer factors itself and 1).
The difficult part of obtaining the factors of a really big number compared to the easy multiplication of its creation ($P \cdot Q \rightarrow N$ but given $N \nrightarrow P, Q$).
- Modular arithmetic. ($1 \equiv 15 \pmod{2} \equiv 37 \pmod{2}$)

TERMS:

- $P \rightarrow$ Prime number 1.
- $Q \rightarrow$ Prime number 2.
- $N \rightarrow = P \cdot Q$ Public modulus for encryption.
- $\varphi(N) \rightarrow$ Euler's totient of N (if P and Q are primes): number of integers $1 \leq x \leq N$ coprime to N . For $N = P \cdot Q \rightarrow \varphi(N) = (P - 1) \cdot (Q - 1)$.
- $E \rightarrow$ made by $\gcd(E, \varphi(N)) = 1 \leftrightarrow E$ is coprime of $\varphi(N)$ Public exponent for encryption.
- $D \rightarrow D \cdot E \equiv 1 \pmod{\varphi(N)}$ Used to decrypt $M \equiv C^D \pmod{N}$.
- $M \rightarrow$ Plaintext with message block with $0 \leq M < N$.
- $C \rightarrow$ Ciphertext encrypted block, $C \equiv M^E \pmod{N}$.

1. **Choose two large primes P and Q .**

Pick carefully 2 very large distinct primes $P \neq Q$.

- Why $P \neq Q$? $\rightarrow$ Because if $P = Q = p \rightarrow N = p^2$. An attacker can take $\sqrt{N} = p$ and immediately recover p . So $P \neq Q$ or else factoring N is trivial (weak).
- Why P and Q must that large? $\rightarrow$ Because for small primes P/Q factoring a composite $N = P \cdot Q$ is easier than factoring N for large primes.
With small primes an attacker can factor N quickly (trial division, Pollard's rho, ECM, GNFS, ...). Making **P, Q large moves the cost of factoring beyond practical limits.**
- Why carefully choosing P and Q ? $\rightarrow$ If P and Q are **not chosen randomly**, they can be predictable making it **easier to recover** them. Also, if $P \approx Q \rightarrow N$ becomes a **nearly perfect square** and Fermat-style factoring

CYBERSECURITY REPORT – LABORATORY 4

(looks for $\sqrt{N}$) becomes easy. It **must not happen that $P \approx Q$** . And **avoid primes whose $p + 1$ or $p - 1$ factors are small primes** as they can be attacked by Pollard's $p - 1$ and Williams' $p + 1$ very simple given N by great common divisor (gcd), for this, use safe primes $p = 2q + 1$ making impossible this attack.

- Why we need primes? $\rightarrow$ If $N = P \cdot Q$ with P, Q prime then $\varphi(N) = (P - 1) \cdot (Q - 1)$. That lets the key maker computer $D = E^{-1} \pmod{\varphi(N)}$ easily. If **P or Q were not prime (composite), computing $\varphi(N)$ would require factoring N** so we would lose that shortcut.

2. Compute $N = P \cdot Q$.

N is the **public modulus** that creates a finite modular arithmetic world where **exponentiation is reversible only if you know $\varphi(N)$** . And $\varphi(N)$ is easy to get from P, Q but hard to get from N alone.

3. Compute Euler's totient $\varphi(N) = (P - 1) \cdot (Q - 1)$.

Because P, Q are prime, the count of integers $1 \leq x \leq N$ coprime to N are $(P - 1) \cdot (Q - 1)$.

- If P or Q are not prime? $\rightarrow$ Then, we **cannot apply Euler's totient**, we would need to **calculate the prime factors p_i of N** :

If $N = \prod_i p_i^{e_i}$ we use the formula: $\varphi(N) = N \prod_i (1 - \frac{1}{p_i})$

Else the more general way where N is composed of distinct primes r with exponents e_r we use the general formula: $\varphi(N) = N \prod_{r|N} (1 - \frac{1}{r})$.

4. Choose the public exponent E .

We need to **pick an integer E such that $1 \leq E \leq \varphi(N)$ and $\gcd(E, \varphi(N)) = 1 \leftrightarrow$**

E is coprime of $\varphi(N)$. A common exponent is $E = 65537 = 100000000000000001_2$

5. Compute the private exponent D .

Find D satisfying $D \cdot E \equiv 1 \pmod{\varphi(N)}$. It is calculated with the Extended Euclidean algorithm.

- What is the extended Euclidean algorithm? $\rightarrow$ The **EEA works for any integers, but an inverse exists iff $\gcd(E, \varphi(N)) = 1$** .

The EEA finds the greatest common divisor of 2 numbers and writes it as a combination of them.

Given a, b it finds x, y with: $ax + by = \gcd(a, b)$ and if $\gcd(a, b) = 1$ then x is the **inverse of a mod b** .

How it works the main idea: $a = bq + r, 0 \leq r < b \rightarrow \gcd(a, b) = \gcd(b, r) \dots$ Until the last non $r = 0$.

Ex: $E = 7 \mid \varphi(N) = 40 \rightarrow D \cdot 7 \equiv 1 \pmod{40} \rightarrow 40 = 7 \cdot X + Y \rightarrow 7 = Y \cdot Z + W \rightarrow Y = W \cdot R + T$

6. Encryption.

Given plaintext block M with $0 \leq M < N$, compute Ciphertext $C \equiv M^E \pmod{N}$.

- To calculate $M^E \pmod{N}$ and $C^D \pmod{N}$ (for decryption) **use modular exponentiation** (square-and-multiply).

7. Send ciphertext C .

8. Decryption.

To decrypt compute $M' \equiv C^D \pmod{N}$

- Why $M = M'$? $\rightarrow M' \equiv C^D \equiv (M^E)^D \equiv M^{E \cdot D} \pmod{N}$. As $E \cdot D = 1 + k \cdot \varphi(N)$ then $M^{E \cdot D} = M^{1+k \cdot \varphi(N)} = M \cdot (M^{\varphi(N)})^k$ and with Euler's theorem if M and N are coprime: $M^{\varphi(N)} \equiv 1 \pmod{N}$ so $M \cdot (M^{\varphi(N)})^k \pmod{N} \equiv M \cdot 1^k \equiv M \pmod{N}$

1.2 Generate the following cryptographic keys (Digital Signature/PKI/Generate/Import keys):

- RSA length 512 bits

Generation of an Asymmetric Key Pair

Algorithm

- ☒ RSA
Bit length of RSA modulus: 512
- ☐ DSA
Bit length of DSA prime: 1024
- ☐ Elliptic curves
Identifier (bit length and curve parameter): prime239v1

User data

The key pair will be put in an encrypted PSE with the name shown below. The key pair will be protected by your PIN code.

Last name: PueblaLopez
First name: AlvaroOscar
Key identifier (optional):
PIN: ****
PIN verification: ****

Pin: 1234 – Elapsed Time: 1.772 seconds

- RSA length 1024 bits

Generation of an Asymmetric Key Pair

Algorithm

- ☒ RSA
Bit length of RSA modulus: 1024
- ☐ DSA
Bit length of DSA prime: 1024
- ☐ Elliptic curves
Identifier (bit length and curve parameter): prime239v1

User data

The key pair will be put in an encrypted PSE with the name shown below. The key pair will be protected by your PIN code.

Last name: PueblaLopez
First name: AlvaroOscar
Key identifier (optional):
PIN: ****
PIN verification: ****

Pin: 1234 – Elapsed Time: 1.022 seconds

- RSA length 2048 bits

Generation of an Asymmetric Key Pair

Algorithm

- ☒ RSA
Bit length of RSA modulus: 2048
- ☐ DSA
Bit length of DSA prime: 1024
- ☐ Elliptic curves
Identifier (bit length and curve parameter): prime239v1

User data

The key pair will be put in an encrypted PSE with the name shown below. The key pair will be protected by your PIN code.

Last name: PueblaLopez
First name: AlvaroOscar
Key identifier (optional):
PIN: ****
PIN verification: ****

Pin: 1234 – Elapsed Time: 2.140 seconds

Álvaro Puebla Ruisánchez and Óscar López García

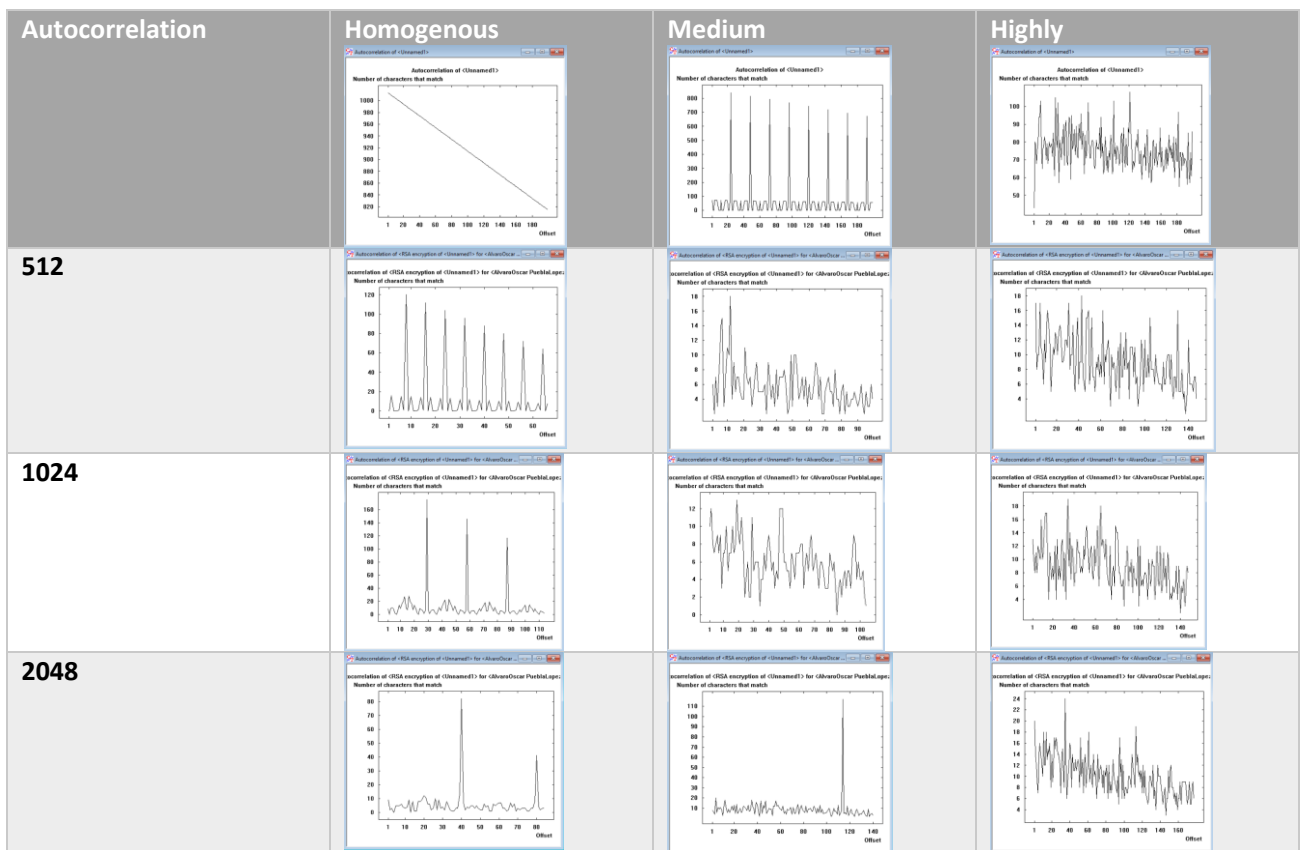
CYBERSECURITY REPORT – LABORATORY 4

fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Phasellus faucibus scelerisque eleifend donec pretium vulputate sapien nec sagittis aliquam. Quisque id diam vel quam elementum pulvinar etiam non quam lacus suspendisse. Nunc lobortis mattis aliquam faucibus purus in massa tempor nec feugiat. Integer feugiat scelerisque varius morbi enim nunc faucibus a pellentesque sit. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vivamus at augue eget arcu dictum varius duis at consectetur lorem.

Curabitur non nulla sit amet nisl tempus convallis quis ac lectus. Sed porttitor lectus nibh. Vivamus magna justo, lacinia eget consectetur sed, convallis at tellus. Praesent sapien massa, convallis a pellentesque nec, egestas non nisi. Nulla porttitor accumsan tincidunt. Pellentesque in ipsum id orci porta dapibus. Mauris blandit aliquet elit, eget tincidunt nibh pulvinar a. Donec rutrum congue leo eget malesuada. Integer ac neque id augue tincidunt suscipit.

Entropy	Homogenous – 0.00/4.70	Medium – 3.80/4.70	Highly – 4.00/4.70
512	5.95/8.00	7.68/8.00	7.88/8.00
1024	6.89/8.00	7.82/8.00	7.85/8.00
2048	7.46/8.00	7.75/8.00	7.86/8.00



CYBERSECURITY REPORT – LABORATORY 4

1.4 For different RSA algorithm key lengths, measure the encryption and decryption time for files of 1MB, 2MB, 5MB (with an accuracy of 1 seconds):

Encrypt	1MB	2MB	4MB
512	0.292	0.601	1.055
1024	0.609	1.050	2.029
2048	1.057	1.965	8.929

Decrypt	1MB	2MB	4MB
512	3.730	17.446	43.917
1024	31.065	22.085	42.942
2048	112.021	71.596	143.877

1.5 Encrypt and decrypt the plaintexts from point 4 using a symmetric algorithm, try to notice the time of these operations:

When encrypting and decrypting the 1 MB, 2 MB, and 5 MB plaintexts using a symmetric algorithm (such as AES), the operations **took less than one second in each case**.

The difference between encryption and decryption times was negligible, especially when compared to RSA, which required several seconds or even minutes for the same files.

1.6 Introduce the following changes to the ciphertext (use a different length of a key). Then, decrypt the ciphertexts and check data integrity:

- change the value of 1 byte,

000001B0	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001BC	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001C8	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001D4	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001E0	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001EC	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000001F8	61 61 61 61 61 61 61 61 61 61 61 61	aaaaa..@8v...
00000204	FD 1C F7 9F E5 95 C1 C3 6B AD AE F4	k...
00000210	0C 34 21 A6 B6 F1 20 2F E6 95 04 76	.4!... /...v
0000021C	90 42 7F 7C 9B 78 4E D4 4A F8 0A CC	.B. .xN.J...
00000228	75 37 2B 01 8C 82 E4 C1 87 FE 11 E5	u7+.....
00000234	5B B7 EB B5 39 B1 2B 07 S7 2A A9 SA	[...9.+..W*.Z
00000240	92 B4 9B 8F 9A D0 70 65 49 FA 1E 88	peI...
0000024C	4A 6C 13 46 E6 83 DF F1 A8 01 0A 82	Jl.F.....
00000258	BE 34 5B 36 78 F0 B6 19 1F 3E 66 82	.4[6x.....>f.
00000264	E5 93 5F DB SE 96 7E 89 D2 8A CE 8A	^.....
00000270	FB 9D 81 14 88 59 9C 7B D7 E0 31 61	Y. {...la
0000027C	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
00000288	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
00000294	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002A0	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002AC	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002B8	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002C4	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002D0	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa
000002DC	61 61 61 61 61 61 61 61 61 61 61 61	aaaaaaaaaaaa

It changes only the corresponding block.

- [illegible]

[illegible]

```

C:\Users\Unamed1> rsa /encrypt /key:2048 /plaintext:Unamed1 /out:Unamed1.enc
C:\Users\Unamed1> rsa /decrypt /key:2048 /ciphertext:Unamed1.enc /out:Unamed1.dec
C:\Users\Unamed1> rsa /encrypt /key:2048 /plaintext:1mb.txt /out:1mb.txt.enc
C:\Users\Unamed1> rsa /decrypt /key:2048 /ciphertext:1mb.txt.enc /out:1mb.txt.dec

```

7

CYBERSECURITY REPORT – LABORATORY 4

1.7 Does the key length affect the ciphertext entropy? If so, how?

Yes, as when we increase the key length, we **also increase the randomness of the ciphertext**, so the entropy is nearer to 8.00. We can see it clearly with the homogeneous plaintext, as when we encrypt it with the different keys, these are the results: 512-> 5.95/8.00, 1024 -> 6.89/8.00, 2048 -> 7.46/8

1.8 Does the length of the key affect the autocorrelation of the ciphertext? If so, how:

Yes, as we **increase the size of the key, the number of spikes is reduced**, so it means that there are less repetitions and the ciphertext is more random.

1.9 Does the ciphertext entropy depend on the plaintext entropy? If so, how?

Yes, when the entropy of the plaintext is **lower**, the entropy of the ciphertext is **lower too**.

1.10 How does the encryption/decryption time depend on the file length?

As we **increase the file length**, the **encryption** time also **increases**, the same with the **decryption**.

1.11 What is the time of encryption/decryption with asymmetric algorithms compared to the execution of these operations with a symmetric algorithm?

The **symmetric** algorithms are **much faster** than the **asymmetric** ones.

1.12 What do the changes in the plaintext look like when you introduce the ciphertext falsification? What is their scale? What does it depend on?

If you remove a byte or more, the block in which the byte is changed and the ones that go after that block will be damaged. If you change one byte or more, the blocks in which you change the bytes will be damaged. Finally, if you remove one entire block, the rest of the blocks will be decrypted correctly, the only change is that the block that you removed is not there.

CYBERSECURITY REPORT – LABORATORY 4

1.13 Is it possible to remove a fragment from the ciphertext so that the remaining text of the plaintext after decryption is readable?

Yes, you must **remove a block of the same size of the key**.

1.14 What are the advantages and disadvantages of asymmetric algorithms compared to symmetric ones?

FEATURE	SYMMETRIC ENCRYPTION	ASYMMETRIC ENCRYPTION
# OF KEYS	1 shared key	1 public key and 1 private key
SPEED TO ENCRYPT/DECRYPT	Fast	Slower
RESOURCE USAGE	Low	High
KEY DISTRIBUTION	Tough (requires secure channel)	Easy (only share public key)
SECURITY	Vulnerable if key compromised	Secure if private key safe
DATA SIZE	Large (files, databases)	Small (keys, emails)
AUTHENTICATION & NON-REPUDIATION	Limited as cannot support verifiability of the message	Supports digital signatures and non-repudiation
KEY LENGTH	128/192/256-bit (AES)	2048-bit or more (RSA)
COMMON ALGORITHMS	AES, DES, 3DES, RC4	RSA (factoring large numbers), ECC (elliptic curve discrete log), DSA (discrete log)
USE CASES	Data at rest, VPNs and disk encryption	SSL/TLS handshakes, digital signatures and Public Key Infrastructure

In summary, **symmetric encryption** uses a **single shared key**, is **fast** and **low-cost** in resources, and is best for large-volume data (e.g., disk or VPN encryption) but **needs a secure channel to share keys**.

Asymmetric encryption uses a **public/private key pair**, is **slower** and more resource-heavy, **simplifies key distribution** and **enables** authentication/non-repudiation (**digital signatures**), so it's suited for small data tasks like SSL/TLS handshakes and signing. Common choices: AES (symmetric, 128/192/256-bit) and RSA/ECC/DSA (asymmetric, typically 2048-bit+ or elliptic-curve keys).

1.15 In which applications is it better to use asymmetric algorithms and in which applications is it better to use symmetric algorithms?

Use asymmetric for **authentication, key exchange and small data**.

Use symmetric for **bulk/fast encryption**.